

Project overview

We want to build a machine-learning-based song recommender that can outperform Spotify's built-in recommendations. The system should accept one seed song and return a ranked list of songs that fit well with that seed. As your mentor, I will help you define the tasks, identify data sources and walk through data collection, cleaning and modelling. The project will initially focus on **R&B/rap** songs to reduce scope and will then generalize to other genres.

Key considerations and constraints

- **Licensing/terms of use** – Spotify's Web API provides detailed audio features (danceability, energy, key, tempo, valence, etc.), but its terms forbid using *Spotify content to train a machine-learning or AI model* ¹. We can use Spotify's API for evaluation or as a feature reference, but we cannot train a model on Spotify's data. Instead we should rely on open-licensed datasets such as **AcousticBrainz**, **Last.fm**, **TheAudioDB** and other open datasets.
- **Free data sources** – We should prioritise APIs that offer free access. Last.fm and TheAudioDB both provide free tiers (with rate limits), and AcousticBrainz's data is CC0-licensed. Some functions in Napster or Deezer may require registration or a paid tier and are therefore lower priority.
- **Cross-matching** – Different APIs use different identifiers. AcousticBrainz indexes tracks by the **MusicBrainz ID (MBID)** and stores low-level and high-level features; Last.fm returns MBIDs in many responses, which we can use to query AcousticBrainz ² ³. TheAudioDB provides endpoints based on TADB IDs and MBIDs ⁴. Designing a robust pipeline will involve matching songs across these services using track title, artist and MBID.
- **Genre focus** – We will start with R&B/rap and later expand to other genres. Last.fm's tag-based API makes this possible: the `tag.getTopTracks` endpoint returns the top tracks for a given genre tag (e.g., "rnb" or "hip-hop") ³. Once the pipeline is in place, we can supply additional tags to fetch other genres.

API and dataset research

Spotify Web API (for reference and evaluation)

- **Audio features** – Spotify's `Get Track's Audio Features` endpoint returns numerical descriptors such as danceability, energy, key, mode, speechiness, tempo, time signature and valence ⁵. These features describe how suitable a track is for dancing, perceptual energy, key signature, tempo (BPM) and emotional positivity. Examples include danceability values between 0 and 1 and tempo in beats per minute ⁵.
- **Recommendations endpoint** – Spotify's `Get Recommendations` endpoint generates track recommendations based on seed songs, artists or genres and allows filters such as minimum or target tempo, danceability, energy, etc. ⁶. However, the policy explicitly states that **Spotify content may not be used to train machine-learning or AI models** ¹, so our model cannot ingest their data.

Last.fm API

API/endpoint	Purpose	Key fields (free tier)	Notes
<code>tag.getTopTracks</code> ³	Returns the top tracks for a given tag (genre). Useful for gathering seed lists of R&B/rap tracks.	Parameters: <code>tag</code> (required), <code>limit</code> (optional), <code>page</code> (optional). Returns track names, artist names, MBIDs and URLs.	No authentication required but an API key is needed. Rate limits apply.
<code>track.getInfo</code> ⁷	Provides metadata for a track given artist/track name or MBID – includes duration, listeners, playcount and top tags.	Parameters: <code>track</code> , <code>artist</code> or <code>mbid</code> . Returns track name, artist, album, duration, listeners/ playcount and top tags.	Useful for enriching metadata and obtaining MBIDs.
<code>track.getSimilar</code> ⁸	Returns similar tracks based on Last.fm listening data.	Parameters: <code>track</code> , <code>artist</code> or <code>mbid</code> , optional <code>limit</code> . Sample responses include similar track names, match scores and artist names ⁹ .	Can be used for evaluation (ground truth) or as additional training signal.
<code>tag.getTopArtists</code> / <code>tag.getTopAlbums</code> / <code>tag.getTopTags</code>	Provide lists of artists or albums associated with a tag.	Accepts <code>tag</code> and optional <code>limit</code> .	Helpful if you later want to expand to artist-based recommendations.

Why Last.fm? Last.fm's API is free for non-commercial projects and returns MBIDs that allow cross-reference with AcousticBrainz. The tag endpoints let us gather R&B/rap songs easily, and the similar-track endpoint provides implicit recommendation data.

AcousticBrainz

- **Project summary** – AcousticBrainz collected low-level acoustic descriptors and high-level features (genre, mood, key, tempo, etc.) from millions of songs between 2015 and 2022 using the open-source **Essentia** toolkit ¹⁰. The data is organised by MusicBrainz recording ID (MBID) and is licensed under **CC0** (public domain) ¹¹.
- **Low-level data** – For each MBID, AcousticBrainz provides a JSON document with spectral descriptors, rhythm features (including beats per minute), tonal information (key and scale) and more ¹². These features are stable and rarely change.
- **High-level data** – High-level features are inferred by pre-trained classifiers from low-level data. They include **mood**, **genre**, **vocals**, **music type** and **danceability** ¹³. For example, the high-level JSON

contains probabilities and a categorical value for danceability (danceable/not_danceable) and other models like gender of vocals and multiple genre classifiers ¹⁴. This data is less stable but very useful for modelling.

- **Access** – Given a recording MBID, you can fetch two JSON documents (low-level and high-level). The MBID must be obtained from MusicBrainz or Last.fm. Because the data is CC0, it can be freely used for training models.

TheAudioDB free API

- **Purpose** – A community-maintained database offering music metadata and artwork. The free API can be used in testing or development. A test key of `2` provides basic access, with a rate limit of **2 calls per second** ¹⁵.
- **Endpoints** – The API includes endpoints to retrieve artist details (`artist.php?i={artistid}`), album details (`album.php?m={albumid}`), track details (`track.php?h={trackid}`), and search endpoints by artist or track name ⁴. It does **not** provide audio features like tempo or mood, but it can supply metadata such as track length and album information.
- **Identifiers** – Supports TADB IDs and MusicBrainz IDs for cross-linking ⁴.

Additional datasets/APIs

- **Napster, Deezer, Amazon Music, Apple Music (MusicKit API)** – Offer rich metadata and streaming capabilities, but typically require registration and may have licensing restrictions. Napster's API provides access to metadata and user playlists ¹⁶. Deezer's API allows unlimited unauthenticated GET requests (according to secondary sources), but the official documentation was not accessible from this environment. These APIs could be explored once the core system is working.
- **Million Song Dataset** – A large, freely available collection of audio features and metadata for one million songs. However, the official site was inaccessible. It is known to provide features like loudness, pitches and timbre at the segment level and could be used if accessible.

Data collection strategy

1. **Define genre seeds.** For R&B/rap, identify relevant tags such as `rnb`, `rhythm and blues`, `hip-hop` and `rap`. Use Last.fm's `tag.getTopTracks` endpoint to fetch a list of popular tracks for each tag. Set the `limit` parameter (e.g., 500) and iterate through `page` values to gather thousands of tracks.
2. **Retrieve metadata and MBIDs.** For each track returned, call `track.getInfo` to obtain the MusicBrainz recording ID (MBID) as well as listeners, playcount and top tags ⁷. Store the track name, artist and MBID.
3. **Fetch audio features.** Using the MBID, query AcousticBrainz for the low-level (`/low-level.json`) and high-level (`/high-level.json`) data. Extract features such as tempo (BPM), key, mode, danceability probability/value, mood probabilities (happy, sad, relaxed, etc.) and genre predictions ¹³ ¹⁴.
4. **Optional metadata enrichment.** Use TheAudioDB to fetch additional metadata (album names, release years) by querying endpoints like `track-mb.php?i={mb_recording_id}` ¹⁷. This step is not mandatory for the recommendation model but could be valuable for user-facing information.
5. **Gather implicit recommendation data.** For evaluation and training, call `track.getSimilar` on a subset of seed tracks to obtain lists of similar tracks and their match scores ⁹. This can act as a pseudo-ground-truth for training or validating a recommender.

6. **Store the data.** Design a relational (e.g., PostgreSQL) or document-oriented (e.g., MongoDB) database with tables/collections for tracks (track_id, title, artist, MBID), audio_features (MBID, tempo, key, mode, danceability, mood features, genre vectors), tags (list of Last.fm tags) and similar_tracks (seed_track_id, similar_track_id, match_score). Index on MBID for fast joins. Persist API responses to avoid repeated calls.

Data cleaning and preparation

1. **Deduplicate and normalize.** Normalize artist and track names (lowercasing, removing punctuation) and deduplicate records. Use MBID as the primary key when available; if missing, create a composite key using normalized artist and track name.
2. **Handle missing data.** Some tracks may lack high-level features or MBIDs. Decide whether to drop these records or impute values (e.g., using mean tempo of the artist). Document any imputation strategy.
3. **Feature engineering.**
4. Convert categorical features (e.g., predicted genre categories, mood labels) into numerical representations. High-level JSONs provide probability distributions; these probabilities can be used directly as features.
5. Standardize numeric features such as tempo, loudness, energy and valence to have zero mean and unit variance.
6. Create additional features like **playcount** and **listeners** from Last.fm to reflect popularity and listener engagement.
7. If you later incorporate lyrics (from a legally licensed dataset), compute sentiment scores or embeddings, but avoid scraping unauthorized lyrics due to legal restrictions ¹⁸.
8. **Partition the dataset.** Split the data into training, validation and test sets. Because recommendation is about ranking, ensure that songs from the same artist or album do not appear across splits to avoid leakage.

Modelling approach

1. **Baseline: content-based nearest neighbours.** Compute feature vectors for each track using the engineered features. Use cosine similarity to find nearest neighbours for a seed song. This simple model serves as a baseline and is interpretable.
2. **Matrix factorization or collaborative filtering.** If you accumulate user-track interaction data (e.g., from Last.fm scrobble logs in the future), use matrix factorization (e.g., Alternating Least Squares) or implicit feedback models like LightFM. For now, you can treat the similar-tracks lists from Last.fm as implicit interactions and factorize a *track-track* similarity matrix.
3. **Graph-based models.** Build a graph where nodes are tracks and edges connect similar tracks (from Last.fm) or tracks with high feature similarity. Apply graph neural networks or random walk algorithms (e.g., DeepWalk, Node2Vec) to learn embeddings and recommend tracks.
4. **Neural networks.** For a more powerful model, train a neural network (e.g., Siamese network) that takes two song feature vectors and predicts similarity. Use pairs from Last.fm similar-track lists as positive examples and random pairs as negatives.
5. **Evaluation.**
6. Use metrics such as **Precision@k**, **Recall@k**, **Mean Average Precision (MAP)** and **Normalized Discounted Cumulative Gain (NDCG)** to compare your model's recommendations against ground-truth similar-tracks lists.

7. Perform offline experiments for each genre and compare performance to Spotify's `Get Recommendations` endpoint results. Because Spotify's content cannot be used for training, evaluation should only compare ranking outcomes.
8. Later, conduct user studies or A/B tests (if you have access to listeners) to gather subjective feedback.

Extending to other genres

Once the R&B/rap pipeline is running:

1. **Add new tags.** For each additional genre (e.g., rock, pop, jazz), call `tag.getTopTracks` with the genre tag and repeat the data collection process. Because the pipeline uses MBIDs, the same AcousticBrainz feature extraction applies.
2. **Update the model.** Retrain or fine-tune the model with the expanded dataset. Consider weighting by genre to avoid bias toward more popular genres.
3. **Multi-genre handling.** Tracks often belong to multiple genres. Use the high-level genre probabilities from AcousticBrainz as features and allow the model to learn cross-genre similarities. Provide a genre parameter in the recommendation API so users can filter results.

Task list / roadmap

The following is a high-level task list. Each task can be further broken down into subtasks. Dates are suggestions; adjust based on your pace.

Phase	Tasks	Description
Phase 1 – Project setup (1–2 weeks)	1. Requirements & environment	Define project scope, evaluation metrics and legal constraints. Set up version control (Git) and choose programming environment (Python). Register for Last.fm and TheAudioDB API keys.
	2. Database design	Design a relational schema for tracks, audio features, tags and similar tracks. Set up a local PostgreSQL or SQLite database and create tables.
Phase 2 – Data collection (2–3 weeks)	3. Genre seed collection	For R&B and rap tags, use <code>tag.getTopTracks</code> to fetch lists of tracks ³ . Store track name, artist, MBID and tag.
	4. Metadata enrichment	For each track, call <code>track.getInfo</code> to obtain MBID, listeners, playcount and album information ⁷ . Handle missing MBIDs by using normalized artist and track names as a fallback key.
	5. Audio feature extraction	Query AcousticBrainz using MBIDs to download low-level and high-level JSON. Extract tempo, key, mode, danceability, mood probabilities and genre probabilities ¹³ ¹⁴ .

Phase	Tasks	Description
	6. Save raw data	Insert all retrieved data into the database. Log API calls to respect rate limits and avoid duplicate requests.
Phase 3 – Data cleaning & feature engineering (2 weeks)	7. Normalize and deduplicate	Clean artist/track names, remove duplicates, and ensure each track has a unique identifier (prefer MBID).
	8. Handle missing values	Decide how to handle tracks missing certain features. Impute or drop as appropriate.
	9. Feature engineering	Create numeric feature vectors: standardize tempo, energy, valence; convert categorical high-level predictions into probabilities. Add popularity features from playcount/listeners.
Phase 4 – Modelling & evaluation (3–4 weeks)	10. Baseline model	Implement a content-based recommender using cosine similarity on the feature vectors. Test retrieval quality using Last.fm similar-track lists as ground truth.
	11. Advanced models	Experiment with matrix factorization or neural network architectures to learn track embeddings. Use positive pairs from <code>track.getSimilar</code> and negative sampling.
	12. Evaluation	Define evaluation metrics (Precision@k, Recall@k, MAP, NDCG) and compute them on a held-out test set. Compare against Spotify's recommendation results without using Spotify data for training.
Phase 5 – Expansion & deployment (ongoing)	13. Add more genres	Extend the data collection pipeline to additional genres using new tags. Repeat feature extraction and retraining.
	14. Build a user interface	Develop an API or web interface that accepts a seed song, displays recommended tracks and allows filtering by genre or mood.
	15. Continuous improvement	Monitor model performance, collect user feedback, and refine data pipelines (e.g., by incorporating more datasets or updating features as AcousticBrainz models evolve).

Conclusion

By leveraging open-source music data from Last.fm and AcousticBrainz, we can assemble a rich dataset of R&B/rap songs containing metadata, high-level mood/genre predictions and low-level audio descriptors. Careful data cleaning and thoughtful feature engineering will allow us to build baseline recommender models and then explore more sophisticated algorithms. Starting with a single genre simplifies the pipeline,

and once the workflow is proven we can expand to additional genres. Throughout the project, remember to respect licensing constraints (e.g., avoid training models on Spotify content ¹ and refrain from scraping lyrics due to Genius's restrictions ¹⁸).

¹ ⁶ **Web API Reference | Spotify for Developers**

<https://developer.spotify.com/documentation/web-api/reference/get-recommendations>

² ⁷ **API Docs | Last.fm**

<https://www.last.fm/api/show/track.getInfo>

³ **API Docs | Last.fm**

<https://www.last.fm/api/show/tag.getTopTracks>

⁴ ¹⁵ ¹⁷ **Free Music API Guide | TheAudioDB.com**

https://www.theaudiodb.com/free_music_api

⁵ **Web API Reference | Spotify for Developers**

<https://developer.spotify.com/documentation/web-api/reference/get-audio-features>

⁸ ⁹ **API Docs | Last.fm**

<https://www.last.fm/api/show/track.getSimilar>

¹⁰ ¹¹ **AcousticBrainz**

<https://acousticbrainz.org/>

¹² ¹³ ¹⁴ **API - AcousticBrainz**

<https://acousticbrainz.org/data>

¹⁶ **Welcome • Napster API**

<https://developer.prod.napster.com/>

¹⁸ **How It Works — lyricsgenius documentation**

https://lyricsgenius.readthedocs.io/en/master/how_it_works.html